

Cloud Computing

Assignment

Date _____
Page _____

1. Differentiate between Scalability and Elasticity with an example.

⇒ Scalability	Elasticity
1. Ability of a system to handle growth by adding resources, usually planned in advance.	1. Ability of a system to automatically add or remove resource based on real-time demand.
2. Focused on long-term capacity expansion.	2. Focuses on short-term, dynamic resource adjustment.
3. It's more about supporting future growth.	3. It's more about reacting to current demand.
4. May require manual intervention to add resources.	4. Often uses automated scaling policies.
5. Resources are added or removed manually in response to workload changes.	5. Resources are automatically scaled up or down based on demand.
6. Example: adding a new server every month as user grows.	6. Example: adding servers during peak hours and removing them at night.

2. A tech startup needs to run two legacy application servers that require complete isolation from each other, alongside a lightweight web service. Compare Type-1 (Bare-Metal) and Type-2 (Hosted) hypervisors for this scenario. Which hypervisor type would you recommend for production deployment versus local development, and why?

⇒ Type-1 (Bare-Metal)

1. Runs directly on the physical hardware, so it has less overhead and better performance.

2. Gives stronger isolation between virtual machines, which is useful when the servers must stay completely separate.

3. Best for production deployment because it is more stable, secure & efficient for always-on servers.

4. Good choice for two legacy application servers that need complete isolation from each other.

Type-2 (Hosted)

1. Runs on top of a host operating system, so it depends on that OS & adds extra overhead.

2. Isolation is good, but not as strong as Type-1 because the host OS is also part of the system.

3. Best for local development because it is easier to install and use on a personal computer.

4. Good choice for testing the lightweight web service on a developer machine.

I would recommend Type-1 for production and Type-2 for local development. The start-up's two legacy servers need strong isolation and better performance, so Type-1 fits production better. Type-2 is simpler for developers who just want to run and test things logically.

3. A banking institution wants to move its customer-facing mobile application to the public cloud to handle unexpected traffic spikes, but strict regulations forbid storing core transaction logs on public infrastructure.

- Which cloud deployment model (Public, Private, Hybrid, or Community) best fits this scenario? Explain how Service-Oriented Architecture (SOA) principles facilitate this setup.
- Why would a developer team choose AWS Elastic Beanstalk / Google App Engine (PaaS) over managing raw AWS EC2 instances (IaaS) when deploying a time-critical app?

⇒ The best fit is Hybrid Cloud. It lets the mobile app use public cloud elasticity to absorb traffic spikes, while keeping core transaction logs and regulated data on private infrastructure to satisfy compliance requirements. A hybrid setup is especially can live in public infrastructure but sensitive workloads must remain isolated.

SOA breaks the system into independent services with well-defined interfaces. That means the customer-facing app, authentication, payments, & logging can be separated so only the non-sensitive parts are exposed to the public cloud, while the regulated log service stays private. Loose coupling & contract-based communication make this split easier to manage & integrate.

A developer team would choose PaaS when speed matters because it reduces operational work: the platform handles provisioning, deployment, scaling, health checks, and much of the runtime management. With IaaS on EC2, the team must manage more of the stack themselves, which adds setup & maintenance time, PaaS is usually better for a time-critical app that needs fast delivery.

4. Consider processing a 500 GB web log file to count daily unique IP address.

- Why does traditional sequential processing fail for this scale, & how does the Map Reduce programming model achieve parallel efficiency?
- What happens during the shuffle and sort phase between Mapper & Reducer, & why is network

bandwidth often the biggest bottleneck in MapReduce performance?

⇒ Traditional sequential processing fails here because a single machine has limited CPU, RAM, disk throughput, and time; scanning 500 GB on one node is slow & may not fit well in memory. MapReduce solves this by splitting the file into chunks, running Map tasks in parallel across many nodes and then combining results in Reduce tasks, so the work scales horizontally instead of depending on one machine.

During Shuffle & Sort, the intermediate key-value pairs produced by mappers are moved to the appropriate reducers based on key, then grouped & sorted so each reducer receives all values for a given key together. This step is expensive because it requires large data transfers across the cluster, often with extra disk I/O and sorting overhead.

MapReduce reduces computation bottlenecks, but the shuffle phase still needs large amounts of data to travel between machines. When the intermediate data is huge, network transfer becomes slower than the actual CPU work, so bandwidth often limits overall performance.